

API Audit Board

	Auditeur	Audit�
Groupe	ThermoSense	ThermoSense
Membres pr�sents	Samuel L�obon - Eliot Louys	Tommy Angibault - Valentin -Mat�o
Date	18/05/2026	
Contrat audit�	OpenAPI	
P�rim�tre audit�	Ensemble du sch�ma	

Crit res

Grille des 12 crit res

Crit re 1

- **Description** : Naming coh rent - ressources au pluriel, substantifs, pas de verbes dans l'URI (/getUsers X).
- ** valuation (~ / X)** : ok
- **Observations (fait observable)** : Tout au pluriel, pas de verbes dans les URIs (coh rent)
- **Recommandation** : URIs sensors/actuators tr s longs, possible de les r duire

Crit re 2

- **Description** : Verbes HTTP - s mantique respect e (GET lecture, POST cr ation, pas de POST pour lecture idempotente, etc.).
- ** valuation (~ / X)** : ok
- **Observations (fait observable)** :
- **Recommandation** : RAS

Crit re 3

- **Description** : Status codes - distincts, document s par op ration (201 cr ation, 204 sans body, 404/409/422   bon escient).
- ** valuation (~ / X)** : ~
- **Gravit ** : Moyenne (impact sur le debugging)
- **Observations (fait observable)** : Assez limit , beaucoup de 400 ou 404. Seulement deux endpoints significatifs (un 401 sur login et un 409 sur les mesures). Certains codes d'erreur n'ont pas un descriptif coh rent ("b timent inexistant" pour un code 403)

- **Recommandation** : Étendre la liste des status codes pour chaque endpoint

Critère 4

- **Description** : Erreurs structurées - Problem Details (RFC 7807) ou schéma d'erreur réutilisable, pas seulement message libre.
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** : Bon schéma réutilisable d'erreur en JSON, contenant code, message et détails.
- **Recommandation** : RAS

Critère 5

- **Description** : Paramètres path, query, header typés avec contraintes (format, min/max, enum).
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** :
- **Recommandation** :

Critère 6

- **Description** : Schémas champs required explicites, pas d'ambiguïté sur les champs obligatoires.
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** : Les champs required sont présents et cohérents
- **Recommandation** : RAS

Critère 7

- **Description** : Exemples au moins une opération critique avec exemple request et response réalistes.
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** :
- **Recommandation** : RAS

Critère 8

- **Description** : Versioning - stratégie documentée (URI/v1, header, media type) + politique d'évolution.
- **Évaluation (~ / X)** : ~ ok via E-tag pour le versionning de ressource, mais pas pour le versionning global de l'API
- **Gravité** : Modérée
- **Observations (fait observable)** : E-tag présents mais mieux de passer par un header je trouve
- **Recommandation** : Documenter davantage la stratégie de versioning, étendre les headers pour ça

Critère 9

- **Description** : AuthN / AuthZ - securitySchemes, scopes ou rôles, endpoints protégés identifiés.
- **Évaluation (~ / X)** : Ok
- **Observations (fait observable)** : implémentation de rôles et conditionnement des endpoints a certains rôles.
- **Recommandation** : RAS

Critère 10

- **Description** : Idempotence - opérations à effet de bord (commande, paiement, alerte): clé ou sémantique documentée.
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** : toutes les requêtes sont idempotentes
- **Recommandation** : RAS

Critère 11

- **Description** : Pagination / filtrage - si endpoints de liste: limit, cursor/offset, filtres documentés (N/A si pas de liste).
- **Évaluation (~ / X)** : ~
- **Gravité** : basse (limite les fonctionnalités de filtrage côté client)
- **Observations (fait observable)** : pagination ok, filtrage prévu mais pas dans le contrat
- **Recommandation** : préciser les options de filtrage dans le contrat

Critère 12

- **Description** : Limites / rate limiting 429 ou règles d'usage (quota, fair use) mentionnés.
- **Évaluation (~ / X)** : ok
- **Observations (fait observable)** : Vérification par IP, pour pas que même personne envoie 20000 requêtes.
- **Recommandation** : RAS

Points positifs :

- Peu de codes erreurs, mais schéma descriptif d'erreur très détaillé.
- Rôles bien implémentés et chaque endpoint critique bien soumis à AUTHN